

DATA 266- Generative AI

Lab 2 Report

Pair Programming- 15

Chinmayi Karumuri- 017531345

Sai Nikhil Juluri – 017504071

Link to access the lab files: <https://github.com/ckarumuri/Data-266--Lab-2>

Lab Part 1: MultiModal Retrieval-Augmented Generation (RAG)

Approach and Methodology:

1. Data Ingestion and Preparation:

We began by mounting Google Drive in our Colab environment to access the required files. The dataset included a lengthy economic document in PDF format and a separate CSV file containing 11 questions. We used the pdfplumber library to extract textual content, and the pdf2image library to convert PDF pages into images. OCR was performed on image-based tables and figures using pytesseract to ensure all visual content could be indexed and retrieved effectively. Each figure and table was assigned a numeric ID to allow referencing in response generation. These were saved to a local directory structure for downstream processing.

2. Text and Image Embedding:

Once the textual and visual data were extracted, we used pre-trained models to embed them into vector representations. The textual descriptions were encoded using the all-mpnet-base-v2 model from SentenceTransformers. For images, we first generated captions using the nlpconnect/vit-gpt2-image-captioning model. These captions were then passed through the same SentenceTransformer to produce image embeddings, enabling unified retrieval across modalities. This process allowed us to semantically index and compare both types of content using a common embedding space.

3. Vector Indexing with FAISS:

To enable efficient retrieval, we indexed both text and image embeddings using FAISS, an open-source similarity search engine optimized for dense vectors. Cosine similarity was selected as the distance metric. This setup allowed us to query the vector database with an embedded user question and retrieve the most relevant chunks of text and/or image captions, ranked by similarity score. We used the top k (typically k=3) matches for the response generation phase.

4. Retrieval and Generative Answering:

For each question, the system embedded the query and performed a similarity search on the FAISS index to identify the closest matching text and image. We implemented a reranker using the BAAI/bge-reranker-base model to improve the ranking of results by fine-grained semantic comparison. The top-ranked textual content was then passed to the google/flan-t5-large model, a pre-trained LLM, to generate a coherent and contextually relevant answer. If an image (table or figure) was deemed relevant based on similarity score, its ID was also recorded.

5. Output Formatting and Submission:

The final answers were formatted into a CSV file, adhering to the Kaggle competition's submission guidelines. Each row consisted of an ID, the generated text response, and a corresponding image number. If no relevant image was found, the image field was set to 0. The submission was tested with all 11 questions and manually verified for consistency and formatting.

Evaluation and Metrics:

To evaluate the generated responses, we used BERTScore, a semantic similarity metric that compares the generated text with reference answers using contextual embeddings from BERT. This ensured that the generated output was not only grammatically correct but also aligned with the expected information. The overall results indicated that the retrieval process was effective and the generative model produced fluent and context-aware answers. While quantitative scores were not explicitly part of the notebook, the qualitative inspection of the answers validated the pipeline's effectiveness.

Conclusion:

This lab demonstrated our ability to design and implement a MultiModal Retrieval-Augmented Generation (RAG) system capable of handling complex economic queries using both textual and visual content. We successfully integrated multiple open-source tools and pre-trained models to build an end-to-end pipeline that extracts, embeds, indexes, retrieves, and generates answers based on multimodal data. Our approach also met the expectations of a real-world Kaggle competition, ensuring scalability and robustness. Future improvements could include experimenting with cross-attention fusion of retrieved text and image embeddings or fine-tuning the reranker model for domain-specific documents.

Lab Part 2: Fine-Tuning Stable Diffusion for Image Generation

Approach and Methodology:

1. Dataset Loading and Preparation:

The second part of the lab focused on fine-tuning a Stable Diffusion model using a custom dataset consisting of 1500 images and their corresponding captions. The dataset was loaded from

Google Drive and contained two main components: a directory of image files and a text file (captions1500.txt) containing image-caption pairs. After verifying and cleaning the dataset, captions were preprocessed using standard NLP techniques including lowercasing, punctuation removal, and whitespace normalization to ensure clean textual inputs for training. All images were resized and transformed to match the 512x512 pixel input requirement of the Stable Diffusion model.

2. Custom Dataset Class Implementation:

We implemented a custom `torch.utils.data.Dataset` class that mapped each image to its corresponding captions. Each entry in the dataset consisted of an image and a cleaned textual prompt. During initialization, we filtered out corrupt or mismatched entries to ensure data quality. We also applied image transformations such as resizing, normalization, and tensor conversion using torchvision utilities. The dataset class enabled efficient batch processing during model training.

3. Model Initialization and Training Setup:

We utilized the Hugging Face `diffusers` library to fine-tune the Stable Diffusion model. The pre-trained base used was `CompVis/stable-diffusion-v1-5`. We configured the training pipeline to adopt the LoRA (Low-Rank Adaptation) fine-tuning method using the `peft` library, which allowed parameter-efficient updates to the model weights. A `UNet2DConditionModel` was initialized for the denoising task within the diffusion process. The model, optimizer, and learning rate scheduler were moved to the CUDA device to accelerate training.

The training loop consisted of randomly sampling a batch of images and their captions, encoding the text prompts, adding noise to the image latents, and teaching the model to denoise and reconstruct the original image based on the prompt. We applied the Adam optimizer and a linear scheduler for learning rate decay. Loss values were computed using mean squared error between the predicted and actual noise vectors. The model was trained for a set number of steps with logging enabled for periodic progress visualization.

4. Inference and Image Generation:

Once the fine-tuning was completed, the LoRA-adapted model was saved and later reloaded for inference. We generated images from five different text prompts covering a variety of styles and semantic meanings. The output images were diverse and visually coherent, reflecting the underlying content of the prompts. These results demonstrated that the model successfully learned associations between textual concepts and visual features within the domain of the custom dataset.

5. Evaluation Metrics:

To evaluate the quality and relevance of the generated images, we employed two standard metrics: Inception Score (IS) and CLIP Similarity Score. The Inception Score was used to assess the diversity and visual fidelity of the generated samples, where higher scores indicated more

realistic and varied outputs. CLIP Similarity Score, on the other hand, quantified how well each generated image aligned with its corresponding text prompt, using a pre-trained CLIP model to compare embeddings. The model achieved strong scores on both metrics, confirming the effectiveness of our fine-tuning approach.

Conclusion:

Through this lab, we successfully demonstrated the process of customizing a state-of-the-art generative diffusion model for text-to-image synthesis. We implemented a complete training pipeline from dataset preparation to model fine-tuning and evaluation. The use of LoRA significantly reduced computational overhead while still enabling effective adaptation of the model to our dataset. The final results showed that Stable Diffusion can be fine-tuned to capture domain-specific styles and generate high-quality, prompt-aligned images. Future work may explore full-model fine-tuning or training on more diverse datasets to further improve generalizability and style fidelity.

Lab Part 3: Building an Agentic AI Travel Assistant

Approach and Methodology:

In this part of the lab, we developed a modular AI system consisting of autonomous agents that work collaboratively to generate a personalized travel itinerary based on user input. The system is designed to communicate with real-world APIs to fetch information about flights, weather, hotels, and compile all the retrieved data into a unified travel plan. Each agent is responsible for handling one specific aspect of the travel planning process and communicates through a centralized processing flow. The design emphasizes modularity, error handling, and real-time data retrieval to ensure reliability and extensibility.

1. System Configuration and Initialization:

We started by setting up a dedicated configuration class to store all API keys and endpoint URLs. These included the Amadeus API for flight data, OpenWeatherMap for weather forecasts, and RapidAPI-based Travel Advisor endpoints for hotel and destination information. The system imported required libraries such as requests, datetime, json, and others to manage API requests, data parsing, and error handling. Base functionality for each agent was abstracted into a reusable class named BaseProcessor, which provided a consistent method signature and a shared error reporting mechanism.

2. Implementation of API Agents:

We implemented four primary agents. The first was the FlightSearchProcessor, which authenticated using the Amadeus API and retrieved available flight offers based on the user's input, including departure and return dates, source and destination cities, and number of adults. It handled OAuth2-based token management, caching of IATA codes to reduce redundant lookups, and dynamic resolution of city names to airport codes using the Amadeus location API.

The second agent, `WeatherForecastProcessor`, accessed the `OpenWeatherMap` API to gather forecast data for the specified destination and dates. It provided a detailed breakdown of daily weather conditions including temperature, rain, and general sky description. This agent also parsed and structured the weather results to make them easily digestible for the end-user.

The third agent, `HotelSearchProcessor`, connected to the Travel Advisor hotel listing endpoint via `RapidAPI`. It searched for accommodations near the target location, filtered by availability, pricing, ratings, and distance, and returned a curated list of hotels that best fit the user query. The agent included logic to extract relevant fields such as hotel name, price range, and user reviews.

Finally, the `ItineraryPlannerProcessor` acted as the orchestrator. It called the above agents in sequence, collected the outputs, and formatted them into a well-structured travel itinerary. This included a flight summary, a three-day weather outlook, and a list of recommended hotels. It also handled missing or partial results gracefully and provided fallback messages if data retrieval failed.

3. Modular Workflow and Execution:

We structured the code in such a way that each agent functioned independently but shared a uniform interface for interoperability. All processors inherited from the same base class, and their individual `run()` methods were invoked by a central controller based on the user query. The system was tested on various city-to-city queries involving different dates and number of travelers. Throughout this workflow, extensive error logging and exception handling were included to ensure that a failure in one API did not crash the entire process.

4. Testing and Output Generation:

We tested the travel assistant with multiple queries such as planning a trip from Moscow to Singapore between specified dates for a defined number of travelers. The agents returned meaningful and relevant outputs, including valid flight itineraries, weather forecasts, and hotel lists. The final output was printed in a clean, structured itinerary format showing day-wise weather, travel logistics, and accommodation options.

Conclusion:

This lab successfully demonstrated the power of autonomous agents in building real-world intelligent systems. Each agent was designed with modularity and responsibility isolation, enabling parallel development and testing. By integrating multiple external APIs, we developed a system that simulates the core functionality of a commercial travel assistant. The emphasis on configuration management, graceful error handling, and centralized orchestration reflects good software engineering practices. Future work could extend this assistant to support multi-city travel, activity recommendations, or real-time pricing updates.

